



Rapport du projet de Réseaux

Simulation d'un aquarium de poissons

Mohamed Taha SANDI
Mohamed Dyaé CHELLAF
Mohamed Reda ELKHADER
Othmane BOUTAHRI
Mohamed Amine ETTAKI

Table des matières

1	Client	2
1.1	Langage et bibliothèques utilisées	2
1.2	Diagramme UML du Client	2
1.3	Explication des choix techniques	4
1.3.1	Communication réseau asynchrone et sûre	4
1.3.2	Gestion des commandes et des réponses	4
1.3.3	Modèle réactif et lien avec l'interface	4
1.3.4	Affichage adaptatif	4
1.3.5	Saisie utilisateur	4
1.4	Complexité algorithmique	4
1.4.1	Connexion et écoute réseau	4
1.4.2	Gestion des poissons	4
1.4.3	Synchronisation avec l'interface	5
1.4.4	Concurrence	5
1.5	Initialisation du client et configuration	5
1.6	Traitement des commandes : <code>sendCommandAsync</code>	6
1.7	Envoi et synchronisation des commandes : <code>sendCommand</code>	7
1.8	Diagramme de séquence	8
1.9	Aperçu de l'interface client avec un poisson	9
2	Serveur	10
2.1	Présentation générale	10
2.1.1	Structures de Données Principales	10
2.1.2	Gestion des Communications	10
2.1.3	Traitement des Commandes	10
2.1.4	Gestion des Poissons	11
2.1.5	Déconnexion Automatique	11
2.1.6	Nettoyage des Ressources	11
2.2	Choix de l'implémentation	11
2.2.1	Pourquoi le choix de <code>select</code> ?	11
2.2.2	Limites de <code>select</code>	12
2.2.3	Utilisation de <code>strtok</code> dans les fonctions de parsing	12
2.2.4	Logique du mouvement et stockage des positions	13
2.2.5	Analyse critique de l'implémentation du mouvement des poissons : forces et limites	14
2.2.6	Limites imposées par le client : un obstacle à la pseudo-synchronisation	15

1 Client

1.1 Langage et bibliothèques utilisées

Pour construire le client "Afficheur", nous avons utilisé les outils suivants :

- **Langage** : Nous avons choisi le **langage Java**. Nous avons utilisé ce langage car il fournit des outils intéressants pour la communication réseau (packages `java.net.*`, `java.io.*`) et pour gérer plusieurs tâches en même temps (*multithreading* via `java.lang.Thread`).
- **Interface Graphique** : JavaFX a été utilisé pour créer toute la partie visuelle de l'application. Les packages principaux sont `javafx.application`, `javafx.fxml`, `javafx.scene.*`.
- **Communication Réseau** : Pour que le client puisse communiquer avec le serveur Contrôleur sur le réseau, nous utilisons les outils de base fournis par Java pour le protocole TCP/IP (principalement les classes des packages `java.net` comme `Socket`, et `java.io` comme `BufferedReader` et `PrintWriter`).
- **Analyse des Messages Serveur** : Le serveur envoie des messages sous forme de texte (par exemple, la liste des poissons). Pour extraire les informations utiles de ce texte (comme le nom d'un poisson ou sa position), nous utilisons les "expressions régulières" (`regex`), un outil pour trouver des motifs dans du texte. Le package `java.util.regex` est utilisé.
- **Gestion des Tâches Parallèles (*Multithreading*)** : Pour que l'interface graphique reste fluide et ne se bloque pas pendant que le programme attend des réponses du réseau, certaines tâches (comme l'écoute du serveur) sont effectuées en arrière-plan. Cela est réalisé avec les "*Threads*" de Java (`java.lang.Thread`). Pour mettre à jour l'interface graphique depuis ces tâches d'arrière-plan de manière sécurisée, on utilise `javafx.application.Platform.runLater()`.

1.2 Diagramme UML du Client

Le diagramme UML ci-dessous synthétise la structure du client. Il met en avant les principaux modules (*controller*, *model*, *network*) et leurs relations :

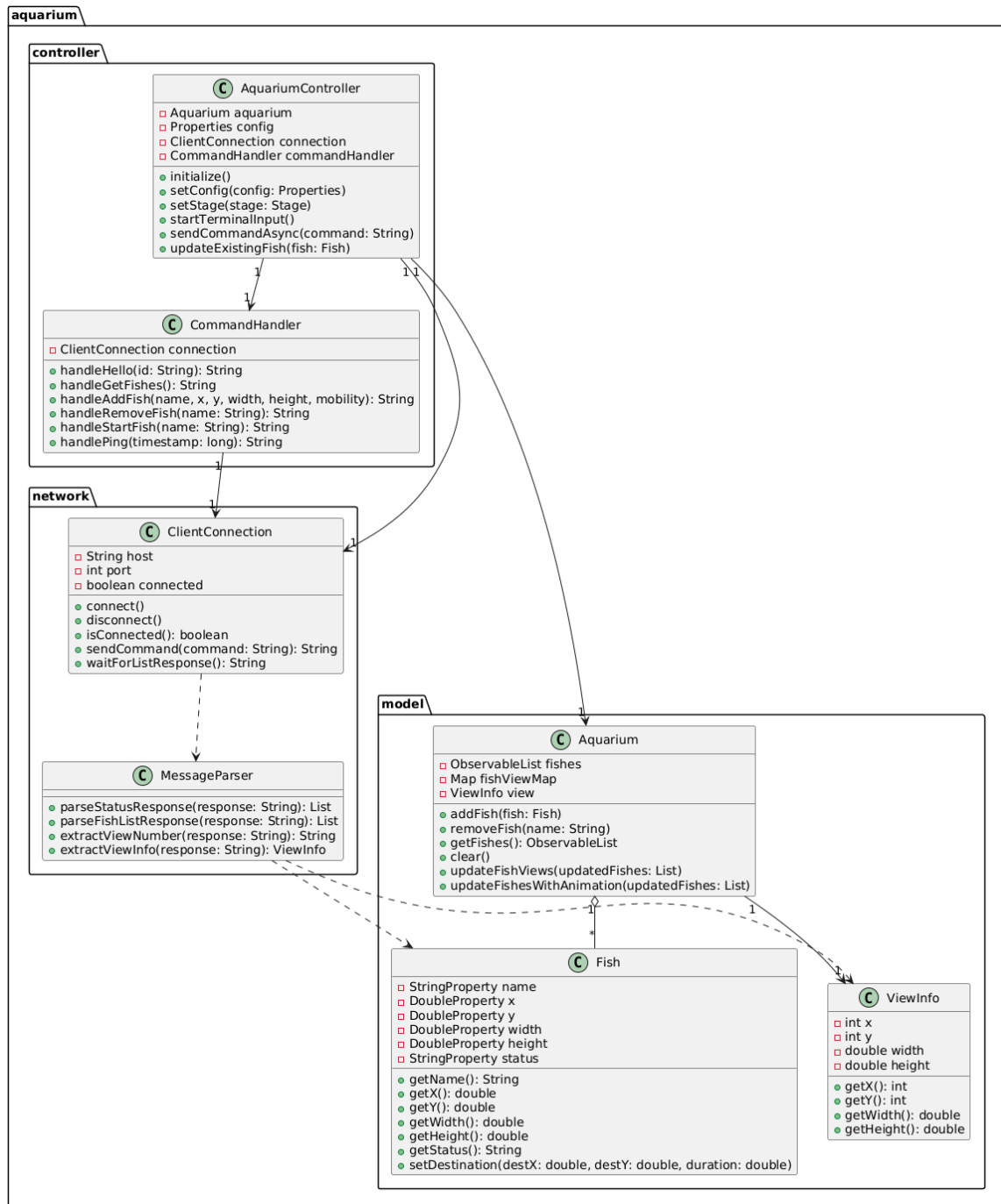


FIGURE 1 – Diagramme UML du client

Description rapide du diagramme :

- **AquariumController** : Point d'entrée de la logique applicative, gère l'initialisation, le dialogue avec le serveur et la synchronisation de l'interface.
- **CommandHandler** : Traite les commandes utilisateur et les traduit en requêtes réseau.
- **ClientConnection** : Gère la connexion TCP, l'envoi et la réception des messages.
- **MessageParser** : Analyse les réponses du serveur.
- **Aquarium** : Représente l'état local de l'aquarium et la liste des poissons.
- **Fish** : Représente chaque poisson (nom, position, taille, statut).
- **ViewInfo** : Informations sur la vue graphique (dimensions, position).

1.3 Explication des choix techniques

1.3.1 Communication réseau asynchrone et sûre

Le client utilise la classe `ClientConnection` pour se connecter au serveur via TCP et gérer cette connexion. Pour que l'application reste fluide, la réception des messages du serveur se fait dans un thread séparé. Ainsi, l'interface utilisateur et le reste du programme ne sont jamais bloqués. Les messages reçus sont mis dans une `BlockingQueue`, ce qui permet à plusieurs parties du programme de communiquer entre elles sans se gêner, en toute sécurité.

1.3.2 Gestion des commandes et des réponses

Toutes les commandes envoyées au serveur passent par la classe `CommandHandler`. Cela permet de bien organiser le code et de rendre les choses plus simples à comprendre et à modifier. Les réponses du serveur sont ensuite analysées par `MessageParser`, qui récupère les informations utiles, comme la liste des poissons ou l'état du serveur.

1.3.3 Modèle réactif et lien avec l'interface

Les objets `Fish` et `Aquarium` utilisent des propriétés spéciales de JavaFX (`StringProperty`, `DoubleProperty`) qui permettent de faire des liens directs avec l'interface graphique. Ainsi, dès qu'une donnée change (comme la position d'un poisson), l'interface se met à jour automatiquement, sans qu'on ait besoin de le faire nous-mêmes.

1.3.4 Affichage adaptatif

Pour que l'affichage fonctionne sur toutes les tailles d'écran, les positions et tailles des poissons sont exprimées en pourcentage de la taille de la vue, grâce à la classe `ViewInfo`. Cela rend l'interface plus souple, plus jolie, et plus facile à maintenir.

1.3.5 Saisie utilisateur

Le contrôleur principal, `AquariumController`, lance un thread dédié pour lire ce que l'utilisateur tape dans le terminal. De cette manière, on peut interagir avec l'application sans jamais bloquer l'interface graphique.

1.4 Complexité algorithmique

1.4.1 Connexion et écoute réseau

Chaque commande réseau est traitée séparément, ce qui permet au programme de rester rapide, même si plusieurs commandes arrivent en même temps. En général, le traitement est proportionnel au nombre de messages, ce qui reste très raisonnable pour un usage classique.

1.4.2 Gestion des poissons

Quand on ajoute, enlève ou met à jour des poissons, on parcourt simplement la liste des poissons, ce qui prend un temps proportionnel à leur nombre ($O(n)$). Avant d'ajouter ou de retirer un poisson, on vérifie simplement s'il est déjà dans la liste.

1.4.3 Synchronisation avec l'interface

Grâce aux propriétés observables de **JavaFX**, on n'a pas besoin de gérer manuellement les liens entre les données et l'affichage : tout se fait automatiquement. Les animations aussi sont gérées par **JavaFX**, qui s'occupe de les faire tourner de manière fluide et efficace.

1.4.4 Concurrency

L'usage de **BlockingQueue** et de threads permet d'éviter les blocages et les erreurs liées à l'accès simultané aux données. Même s'il y a un léger coût pour gérer les threads, il est très faible par rapport aux avantages obtenus, surtout dans une application classique comme celle-ci.

1.5 Initialisation du client et configuration

Lors du lancement du programme côté client, un ***listener*** est démarré dans le terminal afin d'écouter et traiter les commandes saisies par l'utilisateur. Simultanément, le programme lit un fichier de configuration local permettant de définir les paramètres du client. Un exemple typique de fichier de configuration est donné ci-dessous :

```
# Adresse IP du nœud contrôleur
controller-address = 127.0.0.1

# Identifiant du programme d'affichage
id = N1

# Numéro de port TCP d'écoute du nœud contrôleur
controller-port = 9000

# Valeur par défaut pour la transmission du ping
display-timeout-value = 30

# Répertoire relatif ou absolu où se trouvent les visuels pour les poissons
resources = ./fishes
```

Ces informations sont utilisées pour initialiser la communication réseau avec le contrôleur ainsi que pour préparer l'environnement graphique.

Commande de connexion initiale

La première commande à émettre après lancement est la commande **hello**, qui permet d'établir une connexion avec le serveur. Par défaut, cette commande utilise l'identifiant spécifié dans le fichier de configuration, et envoie un message sous la forme :

```
hello in as N1
```

Il est toutefois possible de spécifier un autre identifiant, par exemple :

```
hello in as N2
```

Le serveur répond ensuite avec un message de type :

`greeting N1 0x0+500+500`

Ce message signifie que le client a été accepté avec l'identifiant `N1` (ou un autre si la préférence initiale était indisponible, comme `N2`), et que la fenêtre graphique doit être positionnée à des coordonnées précises sur l'écran, ici `0x0+500+500`. Ces informations sont immédiatement utilisées pour afficher l'interface graphique du client à l'emplacement spécifié.

Cette séquence assure que chaque client occupe une portion distincte de l'écran, l'interface étant conçue pour se répartir sur une grille de 4 fenêtres.

1.6 Traitement des commandes : `sendCommandAsync`

La méthode `sendCommandAsync(String command)` est responsable de l'analyse, du traitement et de l'envoi des commandes saisies par l'utilisateur à destination du serveur contrôleur. Elle est exécutée dans un nouveau thread afin de garantir la non-blocage de l'interface graphique.

Comportement général

Lorsqu'une commande est saisie dans le terminal, si le client est déjà connecté au serveur, la méthode `sendCommandAsync` est invoquée avec la commande correspondante. Elle commence par vérifier que le gestionnaire de commandes (`commandHandler`) est bien initialisé. Si ce n'est pas le cas, un message d'erreur est affiché, invitant l'utilisateur à se connecter d'abord via la commande `hello`.

Traitement par type de commande

Chaque commande est ensuite traitée individuellement selon sa syntaxe :

- **`getFishes`** : envoie une requête de récupération de la liste des poissons. La réponse est analysée, transformée en objets `Fish`, puis affichée dans l'interface graphique en utilisant `Platform.runLater()` pour garantir l'exécution sur le thread JavaFX.
- **`status`** : demande l'état actuel des poissons. La réponse est utilisée pour mettre à jour les données locales des poissons déjà affichés.
- **`addFish Nom at XxY,WxH,Mobility`** : ajoute un poisson à l'aquarium chez le serveur avec nom, coordonnées, taille et comportement de déplacement. La commande est parsée, validée, puis transmise au serveur.
- **`delFish Nom`** : supprime un poisson existant de l'aquarium. Si la suppression est confirmée par le serveur, elle est également reflétée localement.
- **`startFish Nom`** : demande le démarrage du mouvement automatique d'un poisson nommé chez le serveur.
- **`ping <timeout>`** : envoie une commande `ping` avec un délai spécifique pour tester la connectivité ou le délai de réponse.
- **`getFishesContinuously`** : lance un mode d'écoute continue, dans lequel le client reste connecté et traite les mises à jour continues de l'état des poissons. À chaque mise à jour reçue (`list ...`), les nouveaux poissons sont ajoutés et les anciens sont mis à jour en conséquence, avec animations.
- **`ls n`** : demande n listes de positions du serveur ou 3 listes si n n'est pas fournie.
- **`log out`** : déconnecte le client du serveur en appelant la méthode `disconnect()`.

1.7 Envoi et synchronisation des commandes : `sendCommand`

La méthode `sendCommand(String command)` constitue le cœur du protocole de communication synchrone entre le client et le serveur. Son objectif est double :

1. Envoyer une commande texte au serveur via une socket TCP.
2. Attendre et récupérer uniquement la réponse qui correspond précisément à cette commande.

Envoi sécurisé de la commande

L'envoi de la commande est réalisé à travers le flux `out`, encapsulé dans une section `synchronized` afin d'éviter les interférences entre plusieurs threads qui pourraient écrire simultanément. Cela garantit l'intégrité du message envoyé.

```
1 synchronized (out) {  
2     out.println(command);  
3     out.flush();  
4 }
```

Une exception est prévue pour le mode continu (e.g. `getFishesContinuously`) : dans ce cas, la méthode démarre ce mode, l'active en interne (`continuousMode = true`) puis retourne immédiatement un message de confirmation sans attendre de réponse spécifique.

Mécanisme de réponse

Pour chaque commande (hors mode continu), le client entre ensuite dans une boucle d'attente active sur une `BlockingQueue<String>` appelée `responseQueue`. Cette file contient toutes les réponses reçues du serveur, potentiellement par d'autres threads ou en dehors de l'ordre exact d'envoi.

La méthode applique un mécanisme de *matching* pour déterminer si une réponse donnée correspond bien à la commande envoyée. Le principe est le suivant :

- La méthode récupère les réponses de la file d'attente avec un délai d'attente (`poll(timeout)`).
- Pour chaque réponse, elle vérifie si elle correspond à la commande envoyée, à l'aide d'un ensemble de règles de correspondance spécifiques :
 - `getFishes` doit recevoir une réponse commençant par `list`.
 - `addFish` et `delFish` attendent une réponse `OK` ou `NOK`.
 - `ping X` attend une réponse `pong X` avec correspondance exacte du timestamp.
 - Les autres commandes acceptent toute première réponse reçue comme valide.
- Si la réponse ne correspond pas, elle est **réinsérée dans la file d'attente** pour permettre son traitement ultérieur par une autre commande potentiellement en attente.

Ce mécanisme est à la fois flexible (grâce à l'analyse du contenu des messages) et fiable (grâce à la conservation des messages non consommés). Il permet au client d'envoyer plusieurs commandes consécutivement, tout en garantissant que chaque réponse est associée dynamiquement à la bonne requête, même dans un environnement asynchrone et concurrent.

1.8 Diagramme de séquence

Le diagramme de séquence ci-dessous présente les interactions principales entre un client et le serveur. Il détaille les interactions entre le client et le serveur : de l'ouverture de la connexion (création et connexion de la socket) jusqu'à la fermeture de celle-ci, en passant par les échanges de messages.

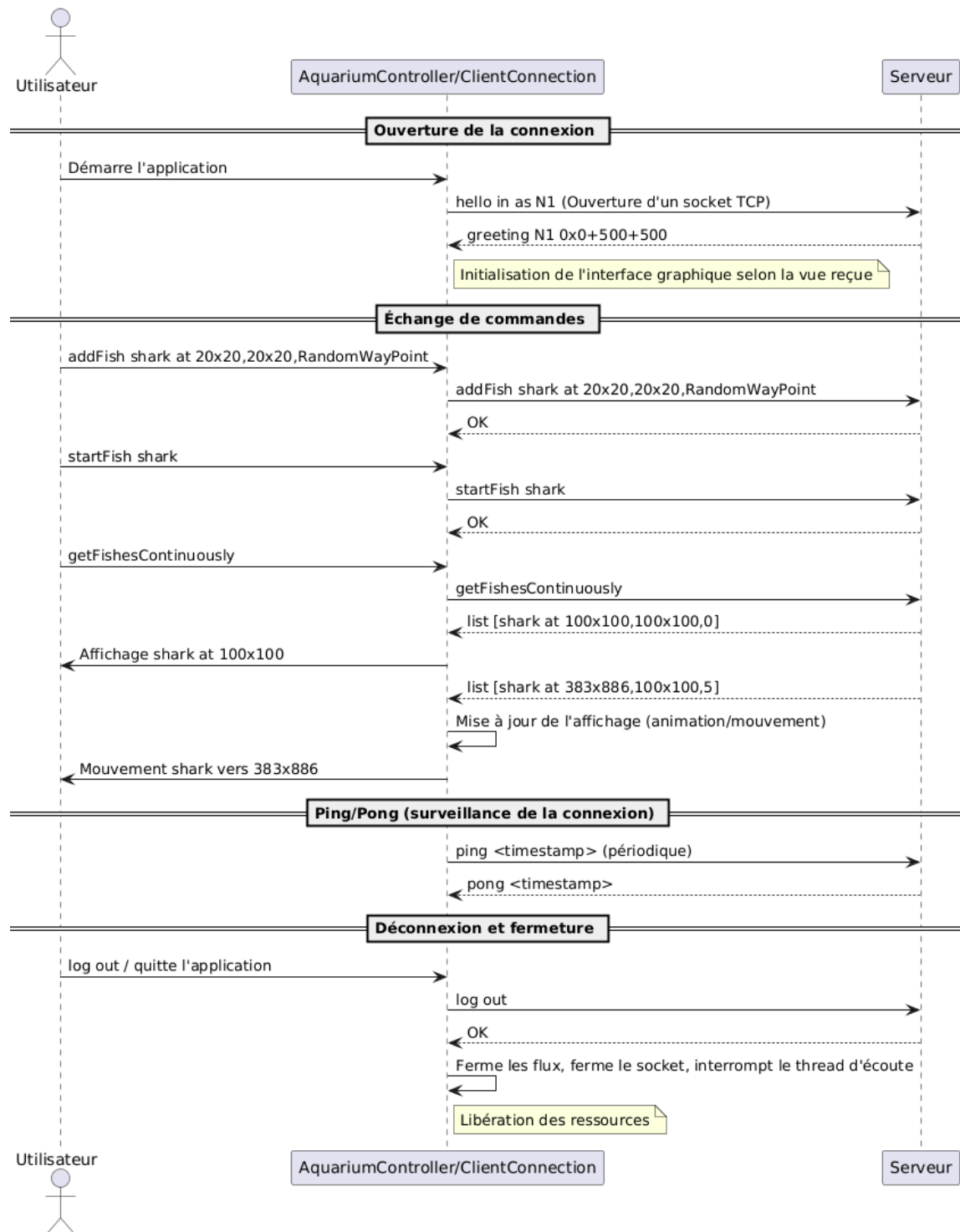


FIGURE 2 – Diagramme de séquence illustrant les interactions entre le Client et le serveur

1.9 Aperçu de l'interface client avec un poisson

La figure ci-dessous présente un aperçu de l'interface graphique d'un client après l'ajout d'un poisson (ici, un requin).



FIGURE 3 – Aperçu de l'interface client avec un poisson

2 Serveur

2.1 Présentation générale

Notre serveur a été entièrement réalisé en langage C, offrant ainsi de hautes performances tout en permettant un contrôle total sur les ressources matérielles. Le démarrage du serveur débute par la création de son propre socket, suivi du lien (*bind*) de ce dernier au port d'écoute spécifié. Une fois le socket correctement configuré, le serveur initialise une liste doublement chaînée destinée à stocker les informations relatives aux différents clients connectés. À ce stade, l'initialisation de l'aquarium est également effectuée.

2.1.1 Structures de Données Principales

Deux structures de données principales sont impliquées dans ce processus :

```
1 typedef struct Aquarium {
2     int width;
3     int height;
4     int active_fishes;
5     int num_views;
6     TAILQ_HEAD(, View) views;
7 } Aquarium;
```

```
1 typedef struct Client
2 {
3     int fd;
4     View *view;
5     time_t time;
6     char *buffer;
7     size_t buffer_len;
8     int continuous_update;
9     TAILQ_ENTRY(Client)
10    entries;
11 } Client;
```

2.1.2 Gestion des Communications

Pour assurer la gestion des communications entre le serveur et les clients, nous avons opté pour l'utilisation de **select** sur un seul *thread*. Cette approche permet une gestion efficace et non-bloquante des échanges réseau, tout en optimisant l'utilisation des ressources.

2.1.3 Traitement des Commandes

Le serveur prend en charge le traitement des commandes envoyées par les clients. Parmi les commandes gérées côté serveur, on retrouve :

Commandes Disponibles

- **load** : Chargement d'un état prédéfini de l'aquarium.
- **del view** : Suppression d'une vue client.
- **add view** : Ajout d'une nouvelle vue pour un client.
- **save aquarium** : Sauvegarde de l'état actuel de l'aquarium.
- **show aquarium** : Affichage de l'état actuel de l'aquarium.

Lorsqu'un client se connecte, le serveur crée une instance associée dans la liste doublement chaînée, puis l'associe à un descripteur de fichier unique pour identifier ses communications.

2.1.4 Gestion des Poissons

La gestion des poissons au sein de l'aquarium est également prise en charge par le serveur, avec une logique de génération de positions et de mise à jour de leurs mouvements dans l'environnement simulé.

2.1.5 Déconnexion Automatique

Un mécanisme de déconnexion automatique est mis en place pour identifier et déconnecter les clients inactifs depuis un certain temps.

2.1.6 Nettoyage des Ressources

Enfin, lors de l'arrêt du serveur, toutes les ressources sont proprement libérées :

- La mémoire allouée pour l'aquarium et les clients est correctement libérée.
- Les descripteurs de fichiers ouverts sont fermés pour éviter toute fuite de ressources.

Cette architecture assure une gestion optimale des ressources tout en maintenant une communication fluide et réactive avec les clients.

2.2 Choix de l'implémentation

2.2.1 Pourquoi le choix de `select` ?

Le choix de `select` dans le cadre de notre projet s'explique par plusieurs raisons stratégiques. Tout d'abord, `select` est une méthode de multiplexage d'E/S largement compatible avec les systèmes Unix/Linux et Windows, ce qui en fait un choix portable et robuste. De plus, l'architecture de notre application est monothreadée, ce qui permet à `select` de gérer efficacement plusieurs descripteurs de fichiers (*sockets*) en parallèle, tout en évitant les problèmes de synchronisation liés au multi-threading. Cette simplicité de mise en œuvre réduit considérablement la complexité du code, tout en assurant un contrôle précis sur les événements réseau (lecture, écriture, erreurs).

Ensuite, dans le contexte du projet, le nombre de connexions simultanées reste raisonnable et bien en deçà de la limite de `select` (1024 descripteurs par défaut), ce qui en fait un choix optimisé.

2.2.2 Limites de `select`

Limite 1 – Nombre de descripteurs maximal

`select` est limité à un nombre fixe de descripteurs de fichiers, généralement 1024 sous Linux (défini par la constante `FD_SETSIZE`). Cette limite peut être augmentée manuellement, mais cela reste peu optimal pour les applications nécessitant une gestion massive de connexions.

Limite 2 – Parcours linéaire des descripteurs

`select` parcourt l'ensemble des descripteurs de fichiers de manière linéaire à chaque appel, ce qui représente un coût en temps de complexité $O(n)$, où n correspond au nombre total de descripteurs surveillés. Cette méthode peut devenir un goulot d'étranglement lorsque le nombre de clients augmente de façon significative. Cependant, dans notre cas, cette contrainte n'est pas problématique puisque le nombre de clients connectés reste relativement faible.

Limite 3 – Coût de copie mémoire

L'appel à `select` implique une copie de l'ensemble des descripteurs entre l'espace utilisateur et l'espace noyau à chaque invocation. Cette copie est coûteuse en mémoire et ralentit le traitement global.

Limite 4 – Absence d'information détaillée

`select` ne fournit aucune information supplémentaire sur l'événement survenu (lecture, écriture ou exception). Il revient au programmeur de parcourir tous les descripteurs pour identifier ceux qui sont prêts, augmentant ainsi la complexité du code.

2.2.3 Utilisation de `strtok` dans les fonctions de parsing

Au début du projet, le choix initial s'était porté sur l'utilisation de `sscanf` pour le parsing des entrées utilisateur dans les différentes parties du code. Cependant, cette approche s'est révélée limitée. En effet, `sscanf` impose la définition de tailles de buffer fixes pour chaque élément détecté dans le motif (*pattern*), ce qui augmente les risques d'erreurs de segmentation (segmentation faults) lors de dépassements de mémoire. De plus, `sscanf` est moins flexible pour le découpage simple de chaînes.

Face à ces limitations, nous avons décidé de migrer vers `strtok`, une fonction plus souple permettant de découper efficacement les chaînes de caractères selon un délimiteur, sans nécessiter de buffers fixes. Cette transition a simplifié le traitement du parsing et réduit les risques liés à la gestion de la mémoire.

Il est toutefois à noter que `sscanf` reste encore utilisé pour la conversion de types (`int`, `float`, etc.) dans certains cas où les motifs sont prédéfinis, étant donné que leur taille est connue à l'avance.

Les fonctions de parsing, tant côté serveur que client, sont chargées de l'analyse, la validation syntaxique, et l'interprétation des commandes textuelles. Chaque fonction ap-

plique un ensemble de règles pour extraire les paramètres, vérifier leur validité, et exécuter l'action attendue (chargement, ajout de vue, manipulation de poissons, etc.). Ces opérations présentent des complexités temporelles dépendant de la taille de l'entrée utilisateur (n), du nombre de vues (k), ou du nombre de poissons (f), selon les cas.

Complexités côté serveur

Fonction	Description	Complexité
<code>is_valid_load_command</code>	Découpe de la chaîne (<code>strtok</code>) + chargement du fichier via <code>parse_aquarium_file</code>	$O(n + m)$
<code>is_add_view</code>	Parsing + vérification d'unicité dans la liste des vues	$O(n + k)$
<code>is_del_view</code>	Parsing + recherche et suppression d'une vue existante	$O(n + k)$
<code>is_save_aquarium</code>	Écriture séquentielle des vues dans un fichier	$O(k)$
<code>is_show_aquarium</code>	Parsing + affichage séquentiel des vues	$O(n + k)$

Complexités côté client

Fonction	Description	Complexité
<code>hello_command</code>	Parsing + recherche d'une vue libre avec <code>get_free_view</code>	$O(n + k)$
<code>status</code>	Parcours et affichage des poissons de la vue associée	$O(n + f)$
<code>add_fish</code>	Parsing + ajout d'un poisson dans chaque vue	$O(n + k)$
<code>del_fish</code>	Suppression d'un poisson (dans chaque vue et chaque liste)	$O(k \cdot f)$
<code>ping_pong, log_out</code>	Parsing simple et réponse immédiate	$O(n)$
<code>get_fishes</code>	Génération de la liste des poissons actifs de la vue	$O(n + f)$
<code>ls</code>	Génère t fois une liste de poissons via <code>generate_list</code>	$O(f \cdot t)$
<code>get_fishes_continuously</code>	Parsing et activation d'un flag de mise à jour	$O(n)$

En résumé, la majorité des fonctions de parsing ont une complexité linéaire par rapport à la taille de l'entrée ou à la structure de données concernée. Les opérations côté serveur agissent sur des entités globales (vues, fichiers), tandis que les commandes client sont centrées sur la vue assignée à l'utilisateur. Cette architecture reste performante dans un contexte de charge modérée, avec un nombre raisonnable de vues et de poissons.

2.2.4 Logique du mouvement et stockage des positions

Dans notre implémentation, la génération des positions d'un poisson débute lorsque le client exécute la commande `startFish`. Ces positions (coordonnées globales) sont stockées dans la structure `Fish`, dont voici un extrait :

```
1 typedef struct Position {
2     int x;
3     int y;
4     TAILQ_ENTRY(Position) entries;
5 } Position;
6
7 typedef struct Fish {
8     char *name;
9     Position *start_position;
10    int width;
11    int height;
12    TAILQ_HEAD(Positions, Position) positions;
13    TAILQ_ENTRY(Fish) entries;
14    int started;
15    char *mobility;
16 } Fish;
```

Listing 1 – Structures Position et Fish

Chaque poisson est dupliqué et stocké dans toutes les vues de l'aquarium. En effet, la structure `View` contient une liste chaînée de `Fish` distincts, comme illustré ci-dessous :

```
1 typedef struct View {
2     char *name;
3     int x, y, view_width, view_height;
4     int taken;
5     TAILQ_HEAD(, Fish) fishes;
6     TAILQ_ENTRY(View) entries;
7 } View;
```

Listing 2 – Structure View

Lorsqu'un client demande une nouvelle position (par exemple via la commande `getFishes`), celle-ci est récupérée depuis la file des positions du poisson correspondant. L'extraction est réalisée en temps constant : la position est retirée de la tête de la file (`TAILQ`) puis libérée immédiatement.

Cependant, comme chaque vue contient sa propre copie indépendante du poisson, une désynchronisation naturelle peut survenir si les clients ne demandent pas tous leurs positions « simultanément ». Dans notre architecture actuelle, la notion de simultanéité n'est pas formellement définie : les demandes sont traitées séquentiellement, client par client, selon l'ordre d'arrivée ou d'itération sur la liste des clients. Ce modèle introduit potentiellement une latence dans la diffusion des mouvements entre les clients, mais celle-ci reste négligeable dans notre contexte, où le nombre de vues et de poissons reste modéré.

2.2.5 Analyse critique de l'implémentation du mouvement des poissons : forces et limites

L'implémentation retenue repose sur une architecture dans laquelle chaque `View` contient sa propre liste de `Fish`, avec une séquence de positions distincte. Ce choix, bien qu'initialement motivé par la simplicité de développement, présente en réalité plusieurs avantages structurants.

Tout d'abord, il offre une isolation complète entre les clients : chaque client interagit uniquement avec les poissons de sa propre vue, ce qui évite toute forme de concurrence d'accès aux données partagées. Cela nous a permis d'implémenter des commandes telles que `ls`, `getFishes` ou `status` de façon locale, sans recourir à des mécanismes de synchronisation complexes (verrous, etc.). De plus, cette organisation garantit une grande modularité : les vues fonctionnent indépendamment les unes des autres, ce qui simplifie la maintenance et facilite le débogage.

Par ailleurs, notre implémentation présente une simplicité conceptuelle précieuse : à chaque requête du client, le serveur sait exactement quoi lui envoyer. Il n'a pas à déduire dans quel état se trouve le client, ni à recalculer une position synchronisée à partir d'un temps global ou d'un historique partagé. Chaque vue possède son propre fil d'exécution logique, ce qui élimine toute forme d'incertitude ou de recalage complexe.

Cependant, cette approche a également ses limites. Le principal inconvénient réside dans la duplication des données : chaque poisson est stocké autant de fois qu'il y a de vues, ce qui augmente la consommation mémoire. Dans notre cas, cette surcharge reste acceptable étant donné le nombre limité de poissons et de clients simultanés, mais elle deviendrait problématique dans un système à grande échelle.

Le serveur ne sait pas si une position envoyée a été effectivement affichée ou si un mouvement a été terminé côté client. Même en centralisant les positions ou en ajoutant un temps global, cette limite subsisterait sans protocole de retour d'état ou contrôle explicite du temps d'affichage. Ainsi, notre architecture favorise la robustesse, la simplicité et la traçabilité dans un environnement asynchrone, au prix d'une synchronisation visuelle entre clients difficile à garantir sans enrichir le protocole de communication.

2.2.6 Limites imposées par le client : un obstacle à la pseudo-synchronisation

Comme précisé dans la spécification, le client n'a pas la capacité de communiquer explicitement la position actuelle des poissons visibles dans sa vue. Cette contrainte rend toute tentative de synchronisation des mouvements entre clients fondamentalement difficile, voire impossible, dans le cas où les connexions ne sont pas simultanées.

Par exemple, si un client se connecte avant un autre, les poissons de sa vue commenceront à se déplacer et consommeront des positions. Lorsqu'un second client se connecte plus tard, le serveur ne dispose d'aucun mécanisme pour savoir à quel instant du mouvement il doit l'insérer dans la "scène" : il ignore quelle position du poisson lui transmettre. Faute d'état partagé. Pour remédier à cette limitation, plusieurs stratégies seraient envisageables :

- **Attendre que toutes les vues soient occupées** avant de commencer la distribution des positions. Cela permettrait une initialisation simultanée des mouvements pour tous les clients connectés.
- **Geler temporairement les poissons** Cette méthode nécessiterait cependant de savoir si un poisson a atteint sa destination ou si le mouvement a bien été exécuté côté client — information absente dans l'architecture actuelle. Néanmoins, comme la durée de chaque déplacement est connue à l'avance, le serveur pourrait en théorie estimer la position du poisson à un instant donné. Cette estimation resterait approximative, car la position réellement affichée dépend aussi du type de trajectoire utilisé côté client (mouvement rectiligne, parabolique, etc.). En l'absence d'une convention partagée ou d'un retour d'état, le serveur ne peut que supposer la progression du poisson, sans garantie d'alignement visuel entre les différentes vues.

Dans notre contexte, la première approche évoquée — qui consiste à ne commencer l'envoi des positions qu'une fois que toutes les vues de l'aquarium sont prêtes à les recevoir — s'avère être la plus pertinente, car elle permet d'assurer un point de départ synchronisé pour l'ensemble des clients.